AI-Native System Design Whiteboard: Refined Scope and Core Direction

Working document v1.2 · Built from PRD Draft v1.0 and the scoping discussion · 11 November 2026

1. Product statement

An accurate, non-hallucinating AI architecture engineer for software system design, useful to developers in any field, built for this one problem and nothing else.

Accuracy comes before features. If the system cannot turn a prompt into a correct, readable architecture, nothing built on top of it matters.

Quality is never traded for cost. The product is built with free tools, but wherever a free option gives better output, the best free option is used.

2. Decisions made so far

These were stated by the product owner and are settled.

- **Focus:** The system must first generate a proper, accurate architecture and diagram from a prompt. Everything else waits until that works.
- **Niche:** The product serves software system design and architecture only. No expansion to other use cases or accessibility features for now.
- **Quality bar:** No compromise on quality. Free tools only, but the best-performing free model is always the one used.
- **Agent lock:** Users cannot alter the agent's work mid-run. The canvas stays restricted until the agent finishes.
- **Local only:** The app runs and is tested on the developer's machine and is never deployed. No production features such as auth, rate limiting, or clustering.
- **Models:** Any free model is acceptable. GLM Flash is the first candidate.
- **Security:** Deferred. First make it exist and work, then harden.
- **Connectivity:** Deferred. Collaboration, import and export, and integrations are not current concerns.
- **Stage:** Planning is done for now. This document goes to the coding agent to start the build.

3. Core pipeline (the only thing to build first)

Prompt, then structured architecture, then deterministic validation, then automatic layout, then render.

Editing existing architectures, AI critique, explanation, and documentation all build on the same structured model and come after this pipeline is reliable.

4. Recommended design principles

These principles guide v1. Revisit them only if the evaluation set shows a problem.

1. **A typed architecture model is the foundation.** The AI produces a graph of components, connections, groups, and properties, not a picture. Validation, editing, diffing, and explanation all read from it.
2. **The AI never decides coordinates.** The model describes structure and meaning. A deterministic layout engine handles position, spacing, routing, and overlap. This is the main lever for clean diagrams.
3. **Apply changes as a single unit.** An agent run either fully lands or rolls back. Because users are locked out during a run, this replaces any need for conflict handling.
4. **Validation is mostly deterministic.** Disconnected nodes, invalid references, and illegal cycles are rule checks. Judgment calls (single points of failure, service boundaries, scalability) are flagged as AI-reasoned critique and kept visibly separate.
5. **One fixed abstraction level for v1.** Either generic components (message queue) or concrete technologies (Kafka). This choice shapes the whole vocabulary.

5. What accuracy means here

For this product, a hallucination is any of the following:

- A component, service, or technology that does not exist or does not fit.
- A connection or data flow that is technically wrong.
- An element with no basis in the requirements, presented as fact.
- A missing dependency that makes the design incomplete.
- Silent acceptance of contradictory or impossible requirements.

Proposed safeguards:

- **Controlled vocabulary.** Components come from a defined set of types, not free text.

- **Schema-constrained output** with automatic validation and retry on failure.
- **Traceability.** Every element links to a requirement, or is explicitly tagged as an assumption or recommendation.
- **Visible assumptions.** The AI states what it assumed rather than hiding it.
- **Ask or flag, do not guess.** Contradictory or underspecified requirements are surfaced to the user.
- **Validate before presenting.** The AI checks its own output and reports problems instead of calling the design complete.

Honest limit: hallucination cannot be driven to zero with free models. The realistic goal is to make errors detectable, visible, and correctable rather than silent.

6. Defining “works”

The definition of success for the first version. A generated architecture counts as proper when:

1. It covers the stated functional and non-functional requirements.
2. Every component is appropriate and every connection is technically correct.
3. Data flows and request paths are accurate.
4. No important dependency is missing.
5. The diagram is clean, aligned, and readable at a glance.
6. Assumptions and trade-offs are stated.
7. It passes all deterministic validation checks.

7. Evaluation from day one

Not a full benchmark, but a small test set from the start.

- Roughly 10 to 20 representative system design prompts, from simple to complex.
- For each, a rubric of what a good answer must contain and must not contain.
- Run after every meaningful change to prompts, schema, or model.
- Track results per model, since reliability on free models is the central risk.

Without this, “it works” is a feeling and every change is a guess.

8. Build baseline for v1

These are the v1 choices, picked to get a working version fast. They were proposed in discussion and adopted at wrap-up. They are the coding agent's starting point.

1. **Abstraction level:** generic component types with an optional technology label (type: message queue, label: Kafka).
2. **Component types (14):** client, API gateway, load balancer, service, worker, database, cache, message queue, object storage, CDN, auth provider, search index, scheduler, external service. These are what a generated diagram may contain, not features of the app.
3. **Connection types:** two only, synchronous and asynchronous, each with a label. Asynchronous loops are allowed; synchronous cycles are flagged.
4. **v1 domain:** web and backend systems (monolith, microservices, event-driven, databases, caching, queues, auth). Widen after accuracy is proven.
5. **v1 scope:** prompt in, architecture out, then the canvas locks. Editing and AI critique come after this pipeline passes its evaluation.
6. **Clarification:** no questions in v1. The AI states its assumptions visibly.
7. **Validation:** deterministic only in v1. It checks for disconnected components, dangling connections, invalid type pairings, synchronous cycles, and missing required links.
8. **Evaluation:** 15 prompts with rubrics, written before the generation code.
9. **Runtime:** local only. Tested on the developer's machine, never deployed.
10. **Models:** any free model through one OpenAI-compatible client with a configurable base URL, key, and model name. GLM Flash is the first candidate and stays unverified until it passes the evaluation set. If a model fails, switch models; do not lower the bar.

Stack

Layer	Choice
Canvas	React + TypeScript + React Flow
Auto-layout	elkjs
Backend	Python + FastAPI
Schema	Pydantic, with JSON Schema generating the frontend types
Graph checks	networkx

Layer	Choice
Models	One OpenAI-compatible client (configurable base URL, key, model name)
Storage	SQLite

Left for the coding agent to settle first: the exact properties each component and connection type carries, and the final model choice from evaluation results.

9. Deferred

Not forgotten, just not now: production deployment and scaling features, security hardening, prompt-injection handling, multi-user collaboration, import of existing diagrams or infrastructure code, export formats, cost estimation, multi-level drill-down views for very large systems, version history and comparison, and the full benchmark.

10. Key risks

- **Free-model reliability** against the accuracy goal. This is the central tension in the project. If a model fails the evaluation, switch models; do not lower the quality bar.
- **Scope creep** from the original eight capability areas. The core pipeline must ship first.
- **Subjective correctness.** System design has many valid answers, so rubrics are needed and judging must not rely on a single biased LLM judge.
- **Open-weight is not the same as open-source.** Model licenses vary and should be checked.

11. Notes for the coding agent

- Planning is done. Section 8 is the v1 baseline; change it only on evaluation evidence.
- Start with the data model and the evaluation set before writing generation code.
- Keep the AI's role to structure and meaning, and keep layout deterministic.
- Keep model providers interchangeable behind a single OpenAI-compatible interface, and pick the model by running the evaluation set.
- Everything runs locally. Do no deployment, auth, rate limiting, or clustering work.
- Never trade accuracy for convenience. If an output cannot be validated, report the problem instead of presenting it as complete.

- Anything outside the core pipeline in section 3 is out of scope until that pipeline passes its evaluation.